

Makina e topave

Madina ka shpikur një makinë topash për të argëtuar pjesëmarrësit e IOI-t. Struktura e brendshme e makinës është si më poshtë:

- Makina përbëhet nga N **nyje**, të indeksuara nga 0 deri në $N - 1$. Nyja $N - 1$ quhet **rrënja**.
- Për çdo nyje u me $0 \leq u < N - 1$, **prindi** i nyjes u është një nyje $P[u]$ me $P[u] > u$, dhe nyja u është **fëmijë** i $P[u]$. Rrënja nuk ka prind. Një nyje pa fëmijë quhet **gjethe**.

Ju **nuk** e dini N dhe as prindin $P[u]$ të asnjë nyjeje u . Në vend të kësaj, Madina ju tregon M , numrin e gjetheve, dhe se indekset e gjetheve janë $0, 1, \dots, M - 1$. Detyra juaj është të përcaktoni strukturën e brendshme të makinës duke e vënë atë në punë.

Çdo nyje e makinës mund të mbajë të shumtën një **top**, dhe çdo top ka një **vlerë**, e cila është një numër i plotë jonegativ. Themi se një nyje ka vlerën x nëse ajo mban një top me vlerë x . Një nyje është **e zënë** nëse përmban një top; përndryshe, ajo është **e lirë**. Fillimisht, çdo nyje është e lirë.

Ju mund të kryeni dy lloje operacionesh:

- **insert**(U, X): perpiquni të shtoni një top të ri me vlerën X atë në gjethen U .
 - Nëse gjethja U është e zënë, operacioni kthen `false` pa e futur topin.
 - Nëse gjethja U është e lirë, topi vendoset në nyjen U . Më pas, topi lëviz në mënyrë të përsëritur të prindi i nyjes së tij aktuale, për sa kohë që ai prind është i lirë. Lëvizja ndalon kur topi arrin rrënjën ose një nyje, prindi i së cilës është i zënë. Operacioni kthen `true`.
- **collect**(): nëse rrënja është e lirë (që do të thotë se makina është bosh), `collect` kthen një varg bosh. Përndryshe, procedura rekursive `traverse`, e përshkruar më poshtë, i mbledh në një varg S vlerat e të gjithë topave që ndodhen aktualisht në makinë. Fillimisht, vargu S është bosh dhe thirret `traverse(N - 1)`.

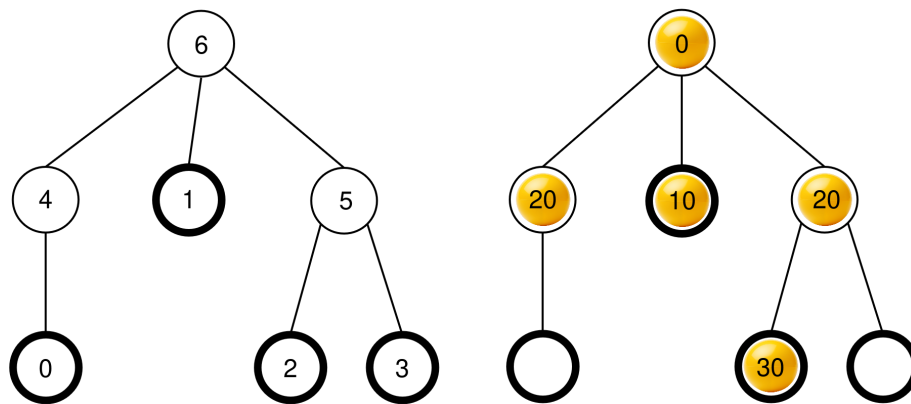
```

traverse(u):
    shto vlerën e topit në nyjen u në fund të  $S$ 
    le të jetë  $c[u]$  lista e fëmijëve të zënë të nyjes u
    rendit  $c[u]$  në rend jozbritës sipas vlerave
        të topave në ato nyje (nëse disa nyje kanë
        të njëjtën vlerë, ato mund të shfaqen në çfarëdo rendi)
    për secilin  $v$  në  $c[u]$ :
        traverse(v)

```

Pasi kjo procedurë përfundon, **të gjithë topat hiqen nga makina dhe collect kthen S .**

Për shembull, shqyrtoni një makinë me $N = 7$ nyje dhe me vargun e prindërve $P = [4, 6, 5, 5, 6, 6]$. Figura në të majtë paraqet makinën bosh me indeksat e nyjeve, ndërsa figura në të djathtë tregon një konfigurim të mundshëm topash pas një sekuence të caktuar operacionesh `insert` (kjo sekuencë jepet në seksionin e Shembullit).



Kur thirret `collect()`, rrënja (nyja 6) është e zënë, kështu që S inicializohet si $S = []$ dhe thirret `traverse(6)`.

traverse(6): nyja 6 ka vlerën 0; duke e shtuar atë në S përftohet $S = [0]$. Fëmijët e nyjes 6 janë 4, 1 dhe 5, të cilët janë të gjithë të zënë. Vlerat e tyre janë përkatësisht 20, 10 dhe 20. Duke i renditur në rend jozbritës përftohet $c[6] = [1, 5, 4]$ (vini re se $c[6] = [1, 4, 5]$ do të ishte gjithashtu e vlefshme, meqenëse nyjet 4 dhe 5 kanë të njëjtën vlerë). Më pas procedura thërret `traverse` për secilën nyje në $c[6]$ sipas atij rendi.

- **traverse(1):** nyja 1 ka vlerën 10; duke e shtuar atë në S përftohet $S = [0, 10]$. Nyja 1 nuk ka fëmijë të zënë, kështu që kjo thirrje përfundon.
- **traverse(5):** nyja 5 ka vlerën 20; duke e shtuar atë në S përftohet $S = [0, 10, 20]$. Nyja 5 ka një fëmijë të zënë, nyjen 2, kështu që $c[5] = [2]$ dhe procedura thërret `traverse(2)`.
 - **traverse(2):** nyja 2 ka vlerën 30; duke e shtuar atë në S përftohet $S = [0, 10, 20, 30]$. Nyja 2 nuk ka fëmijë të zënë, kështu që kjo thirrje përfundon.

- Ekzekutimi kthehet te `traverse(5)`, e cila i ka përpunuar të gjithë fëmijët në `c[5]`, kështu që thirrja e saj përfundon.
- **`traverse(4)`**: nyja 4 ka vlerën 20; duke e shtuar atë në S përftohet $S = [0, 10, 20, 30, 20]$. Nyja 4 nuk ka fëmijë të zënë, kështu që kjo thirrje përfundon.

Të gjitha thirrjet kanë përfunduar dhe nuk ka më nyje për t'u përpunuar. Së fundi, çdo top hiqet nga makina dhe `collect` kthen vargun $S = [0, 10, 20, 30, 20]$.

Detyra juaj është të përcaktoni numrin e nyjeve N dhe të raportoni një varg të prindërve $R = [R[0], R[1], \dots, R[N-2]]$ që përshkruan strukturën e makinës, por me një etiketim potencialisht të ndryshëm të nyjeve që nuk janë gjethe dhe nuk janë rrënja. Formalisht, një varg i raportuar R konsiderohet i saktë nëse është e mundur t'i caktohet çdo nyjeje u të makinës një etiketë unike $L[u]$ me $0 \leq L[u] < N$, e tillë që:

- $L[u] = u$ për të gjitha $0 \leq u < M$ dhe $u = N - 1$, dhe
- $L[P[u]] = R[L[u]]$ për të gjitha $0 \leq u < N - 1$.

Nuk kërkohet që $R[u] > u$. Makina **duhet të jetë bosh** kur raportoni zgjidhjen tuaj.

Le të jetë K numri i operacioneve `collect` të kryera dhe B vlera maksimale e ndonjë top përgjatë të gjitha thirrjeve `insert`. Le të jetë $C = K + B$. Me pas C **nuk duhet ta kalojë 1000**. Pikët tuaja në disa nëndetyra varen nga vlera e C .

Detajet e implementimit

Ju duhet të implementoni procedurën e mëposhtme.

```
std::vector<int> find_structure(int M)
```

- M : numri i gjetheve në makinë.
- Procedura thirret saktësisht një herë për çdo rast testimi.
- Procedura duhet të kthejë një varg $R = [R[0], R[1], \dots, R[N-2]]$ me gjatësi $N - 1$, një varg të saktë të prindërve.

Për të ndërvepruar me makinën, procedura juaj mund të thërrasë dy procedurat e mëposhtme.

Procedura e parë është:

```
bool insert(int U, int X)
```

- U : indeksi i gjetheve ku duhet të futet topi. Kushti $0 \leq U < M$ duhet të plotësohet.
- X : vlera e plotë e topit. Kushti $0 \leq X \leq 1000$ duhet të plotësohet.

- Kjo procedurë kthen `true` nëse topi u fut me sukses, ose `false` nëse gjethja U ishte tashmë e zënë.
- Kjo procedurë mund të thirret të shumtën 500 000 herë.

Procedura e dytë është:

```
std::vector<int> collect()
```

- Mbledh vlerat e të gjithë topave të futur, e zbraz makinën dhe kthen vargun e mbledhur S .

Nëse në çfarëdo momenti gjatë ekzekutimit të programit tuaj vlera e C e kalon 1000, zgjidhja juaj do të marrë verdiktin `Output isn't correct: Too many resources used`.

Struktura e makinës është e fiksuar përpara se të thirret `find_structure`. Sistemi i vlerësimit është **determinist** në kuptimin që, nëse e ekzekutoni dy herë dhe të dyja ekzekutimet kryejnë të njëjtën sekuencë operacionesh, thirrjet e `collect` do të kthejnë të njëjtat vargje (arrays).

Kufizimet

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Nëndetyrat

Nëndetyra	Pikët	Kufizime shtesë
1	5	Rrënja ka saktësisht M fëmijë.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Nuk ka kufizime shtesë.

Në nëndetyrat 3 dhe 4, pikët tuaja varen nga vlera e C si më poshtë.

Nëndetyra 3 (25 pikë)

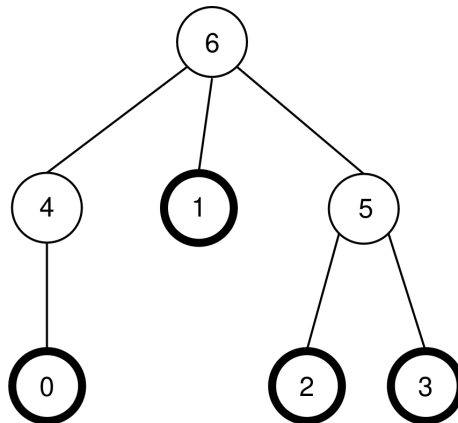
Kufijtë	Pikët
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Nëndetyra 4 (60 pikë)

Kufijtë	Pikët
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Shembull

Shqyrtoni të njëjtën makinë si në përshkrimin e detyrës, pra $N = 7$, $M = 4$ dhe $P = [4, 6, 5, 5, 6, 6]$. Makina tregohet sërish në figurën e mëposhtme:



Vlerësuesi kryen thirrjen e mëposhtme:

```
find_structure(4)
```

Procedura mund të kryejë sekuencën e mëposhtme të thirrjeve:

1. **insert(0, 0)**: gjethja 0 është e lirë, kështu që një top me vlerë 0 vendoset në gjethen 0. Nyja $P[0] = 4$ është e lirë, kështu që topi lëviz në nyjen 4. Nyja $P[4] = 6$ është e lirë, kështu që topi lëviz në nyjen 6. Meqenëse nyja 6 është rrënja, lëvizja ndalon. Thirrja kthen `true`.
2. **insert(3, 20)**: gjethja 3 është e lirë, kështu që një top me vlerë 20 vendoset në gjethen 3. Nyja $P[3] = 5$ është e lirë, kështu që topi lëviz në nyjen 5. Meqenëse $P[5] = 6$ është e zënë, lëvizja ndalon. Thirrja kthen `true`.
3. **insert(1, 10)**: gjethja 1 është e lirë, kështu që një top me vlerë 10 vendoset në gjethen 1. Meqenëse $P[1] = 6$ është e zënë, topi qëndron në nyjen 1. Thirrja kthen `true`.
4. **insert(2, 30)**: gjethja 2 është e lirë, kështu që një top me vlerë 30 vendoset në gjethen 2. Meqenëse $P[2] = 5$ është e zënë, topi qëndron në nyjen 2. Thirrja kthen `true`.

5. `insert(1, 25)`: gjethja 1 është e zënë, kështu që topi nuk vendoset në gjethen 1. Thirrja kthen `false`.
6. `insert(0, 20)`: gjethja 0 është e lirë, kështu që një top me vlerë 20 vendoset në gjethen 0. Nyja $P[0] = 4$ është e lirë, kështu që topi lëviz në nyjen 4. Meqenëse $P[4] = 6$ është e zënë, lëvizja ndalon. Thirrja kthen `true`.

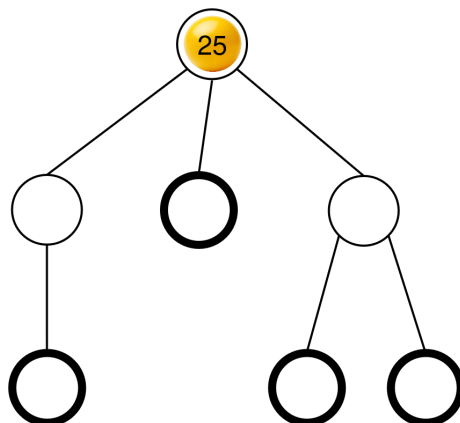
Konfigurimi i përfutur i topave është shfaqur tashmë në përshkrimin e detyrës.

Më pas, procedura mund të thërrasë:

```
collect()
```

Ekzekutimi i kësaj thirrjeje është shpjeguar tashmë në përshkrimin e detyrës, kështu që kjo thirrje kthen $S = [0, 10, 20, 30, 20]$. Më pas, makina është sërish bosh.

Më pas, procedura mund të thërrasë `insert(2, 25)`. Gjethja 2 është e lirë, kështu që një top me vlerë 25 vendoset në gjethen 2. Ky top më pas lëviz në nyjen 5 dhe pastaj në nyjen 6, ku ndalon. Thirrja kthen `true`. Konfigurimi i përfutur i topave tregohet në figurën e mëposhtme.



Së fundi, procedura mund të thërrasë `collect()`, e cila kthen $S = [25]$ dhe e zbraz makinën.

Ka dy vargje të sakta të prindërve që `find_structure` mund të kthejë: $R = P = [4, 6, 5, 5, 6, 6]$ (i cili i përgjigjet etiketimit $L = [0, 1, 2, 3, 4, 5, 6]$) dhe $R = [5, 6, 4, 4, 6, 6]$ (i cili i përgjigjet etiketimit $L = [0, 1, 2, 3, 5, 4, 6]$). Në këtë shembull, $K = 2$ dhe $B = 30$, pra $C = 32$.

Vlerësuesi shembull

Formati i hyrjes:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Formati i daljes:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Këtu, Q është gjatësia e vargut R të kthyer nga `find_structure`.